



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE NINE: ADVANCED PROGRAMMING TECHNIQUES

UNIT ONE: INTRODUCTION TO RECURSION

INTRODUCTION TO RECURSION

Learning Objectives

By the end of this unit, you will be able to:

- **explain the concept of recursion.**
- **develop recursive methods to solve programming problems.**
- **identify and differentiate between base cases and recursive cases in recursion.**
- **analyze and determine situations where using recursion is beneficial.**

Learning Outcomes

At the end of this unit, you should be able to:

- **understand the concept of recursion.**
- **write recursive methods to solve problems.**
- **identify base cases and recursive cases.**
- **recognize problems where recursion is advantageous.**

Introduction

- We explore the concept of recursion, a technique in programming where a method calls itself to solve smaller instances of a problem.
- Recursion is widely used for tasks like traversing data structures (e.g., trees and graphs), solving mathematical problems, and breaking down complex problems into simpler ones.
- We will also cover the key components of recursion, such as the base case (the condition that stops the recursion) and the recursive case (where the function calls itself with modified parameters). Understanding recursion often leads to more elegant and concise solutions compared to iterative methods.

Overview of Recursion

- **Recursion is a programming technique in which a method calls itself in order to solve a problem.**
- **In recursion, a problem is broken down into smaller sub-problems of the same type. These sub-problems are solved independently until they reach a base case, which stops further recursive calls.**

Key Concepts of Recursion

Base and ***recursive*** cases are two key concepts of recursion.

- ***The base case*** is a condition in a recursive function that stops the recursion. Without a base case, the function would continue calling itself indefinitely, leading to a stack ***overflow error***.
- ***The recursive case*** involves the function calling itself, but with a reduced or simpler version of the original problem. The recursive case should always move closer to the base case to avoid ***infinite recursion***.

Example 1: Factorial Calculation

```
public class Factorial {  
    // Recursive Function  
    public static int factorial(int n) {  
        // Base Case  
        if (n == 0 || n == 1) return 1;  
        // Recursive Case  
        return n * factorial(n - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorial(6));  
    }  
}
```

- The ***factorial(int n)*** method checks if n is 0 or 1 (base case), and returns 1. Otherwise, it recursively multiplies n by the factorial of n - 1.
- In the ***main()*** method, ***factorial(6)*** is called, which outputs **720**

How Does Recursion Work

1. Call Stack Visualization

Each recursive call adds a new frame to the call stack until the base case is reached.

Example: factorial(3).

3 * factorial(2)

2 * factorial(1)

1 (base case reached)

2 * 1 = 2

3 * 2 = 6

How Does Recursion Work

1. Call Stack Visualization (continuation)

To find `factorial(3)`, the method calls `factorial(2)`, `factorial(1)` and finally `factorial(0)`. When the base case is reached (`factorial(0)`), the function starts returning values back up the call stack.

`factorial(3) => 3 * factorial(2)`

`factorial(2) => 2 * factorial(1)`

`factorial(1) => 1 * factorial(0)`

`factorial(0) => returns 1 (base case)`

How Does Recursion Work

1. Call Stack Visualization (continuation)

After getting to the base case, values are multiplied back up:

$$\text{factorial}(1) = 1 * 1 = 1$$

$$\text{factorial}(2) = 2 * 1 = 2$$

$$\text{factorial}(3) = 3 * 2 = 6$$

How Does Recursion Work

2. Call Stack Code Simulation

```
public class RecursionStack {  
    public static void recursiveMethod(int n) {  
        if (n <= 0) {  
            System.out.println("Base Case");  
            return;  
        }  
        System.out.println("Recursive Method");  
        recursiveMethod(n - 1);  
    }  
    public static void main(String[] args) {  
        recursiveMethod(3);  
    }  
}
```


How Does Recursion Work

2. Call Stack Code Simulation (continuation)

- The method calls itself with $n - 1$ each time until $n \leq 0$. At that point, it hits the base case, prints "**Base Case**", and begins to return from each previous call.
- This illustrates how the call stack builds up and then unwinds. The above code will output:

Recursive Method
Recursive Method
Recursive Method
Base Case

Examples of Recursion - 1

1. *Fibonacci Sequence*

- The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, typically starting with 0 and 1. The sequences goes: **0, 1, 1, 2, 3, 5, 8,...**

- The formula to find the nth Fibonacci number is:

$$f(n) = f(n - 1) + f(n - 2)$$

Base Cases: if $n == 2$, return 0; if $n == 1$, return 1;

Recursive Case: return $f(n - 1) + f(n - 2)$

Examples of Recursion - 1 (2)

1. *Fibonacci Sequence (continuation)*

```
public class Fibonacci {  
    public static int fibonacci(int n) {  
        if (n <= 1) return n;  
        return fibonacci(n - 1) + fibonacci(n - 2);  
    }  
    public static void main(String[] args) {  
        System.out.println("Fibonacci number of 10: " + fibonacci(10));  
    }  
}
```

- The output of the code is as follows:

Fibonacci number of 10: 55

Examples of Recursion - 2

2. Exponentiation

- Exponentiation is the mathematical operation of raising one number (the base) to the power of another number (the exponent (exp)). The formula to compute the power of a number is:

$$\text{power}(\text{base}, \text{exp}) = \text{base} \times \text{power}(\text{base}, \text{exp} - 1)$$

- **Base Cases:** If $\text{exp} == 0$, return 1 and if $\text{exp} == 1$, return base
- **Recursive Case:** Return $\text{base} \times \text{power}(\text{base}, \text{exp} - 1)$

Examples of Recursion - 2 (2)

2. *Exponentiation (continuation)*

```
public class Exponentiation {  
    public static int power(int base, int exp) {  
        if (exp == 0) return 1;  
        if (exp == 1) return base;  
        return base * power(base, exp - 1);  
    }  
    public static void main(String[] args) {  
        System.out.println("3 raised to power 10: " + power(3, 10));  
    }  
}
```

- The output of the code is as follows:

3 raised to power 10: 59049

Examples of Recursion - 3

3. *Sum of Natural Numbers*

- The sum of the first n natural numbers is the total obtained by adding all numbers from 1 up to n . This can be calculated recursively using the formula:

$$\text{sum}(n) = n + \text{sum}(n - 1)$$

- **Base Case:** When $n = 1$, the sum is 1, so $\text{sum}(1) = 1$.
- **Recursive Case:** For any $n > 1$, the *sum is n plus* the sum of all natural numbers before it, that is $\text{sum}(n - 1)$

Examples of Recursion - 3 (2)

3. *Sum of Natural Numbers (continuation)*

```
public class SumNaturalNumbers {  
    public static int sum(int n) {  
        if (n < 0) return 0;  
        if (n == 1) return 1;  
        return sum(n - 1) + n;  
    }  
    public static void main(String[] args) {  
        System.out.println("Sum of first 5 numbers: " + sum(5));  
    }  
}
```

- The output of the code is as follows:

Sum of first 5 numbers: 15

Examples of Recursion - 4

4. *Sum of Digits*

- The sum of the digits of a number is found by recursively extracting the last digit and adding it to the sum of the remaining digits. This can be calculated using the formula:

$$\text{sumDigits}(n) = (n \% 10) + \text{sumDigits}(n / 10)$$

- **Base Case:** When $n = 0$, sum is 0, so $\text{sumDigits}(0) = 0$.
- **Recursive Case:** For any $n > 0$, the sum is the last digit $(n \% 10)$ plus the sum of the remaining digits $(n / 10)$.

Examples of Recursion - 4 (2)

4. Sum of Digits (continuation)

```
public class SumOfDigits {  
    public static int sumDigits(int n) {  
        if (n == 0) {  
            return 0; // Base case  
        }  
        return (n % 10) + sumDigits(n / 10); // Recursive case  
    }  
  
    public static void main(String[] args) {  
        int number = 12345;  
        System.out.println("Sum of digits of " + number + " is: " + sumDigits(number));  
    }  
}
```

- The output of the code is as follows:

Sum of digits of 12345 is: 15

Examples of Recursion - 5

5. *Greatest Common Divisor (GCD)*

- The GCD of two integers is the largest number that divides both without leaving a remainder. It is can be calculated using the Euclidean Algorithm with the formula:

$$\text{GCD}(a, b) = \text{GCD}(b, a \% b) \text{ where } a \geq b.$$

- **Base Case:** When the remainder ($a \% b$) is 0, the GCD is b.
- **Recursive Case:** For a remainder greater than 0, replace a with b and b with the remainder, then repeat the process until the base case is reached.

Examples of Recursion - 5 (2)

5. Greatest Common Divisor (GCD) (continuation)

```
public class GCD {  
    public static int gcd(int a, int b) {  
        if (b == 0) return a; // Base case  
        return gcd(b, a % b); // Recursive case  
    }  
  
    public static void main(String[] args) {  
        int a = 48;  
        int b = 18;  
        System.out.println("GCD of " + a + " and " + b + " is: " + gcd(a, b));  
    }  
}
```

- The output of the code is as follows:

GCD of 48 and 18 is: 6

Recursion vs Iteration

When to Use Recursion

- i. Recursion is ideal for problems that naturally break down into subprograms, such as tree traversal, factorial, Fibonacci series, etc.
- ii. Recursion tends to have cleaner, more readable code for certain problems.

Recursion vs Iteration (2)

When to Avoid Recursion

- i. Recursion can lead to stack overflow if the recursion depth is too high (e.g. calculating Fibonacci number for large number).**
- ii. Iterative solutions may be more efficient in some cases, especially when working with large datasets.**

In-Text Questions (ITQs)

- 1. What is recursion in programming?**
- 2. What is the purpose of the base case in a recursive function?**
- 3. How does recursion use the call stack to solve problems?**

In-Text Answers (ITAs)

- 1. Recursion is a programming technique where a method calls itself to solve smaller instances of a problem until it reaches a base case that stops the recursion.**
- 2. The base case is a stopping condition that prevents infinite recursion by terminating the recursive calls once a simple, solvable instance of the problem is reached.**
- 3. Each recursive call adds a new frame to the call stack. When the base case is reached, the calls return values back down the stack, resolving the problem step-by-step.**

Conclusion

- **Recursion is a powerful and elegant tool in a programmer's arsenal for solving complex problems that exhibit repetitive or nested structure. By mastering the concept of base and recursive cases, students can apply recursion to a wide range of real-world programming challenges.**
- **However, developers must also recognize the limitations of recursion and know when iterative solutions may be more appropriate. With a firm grasp of recursion, you can tackle problems involving repetition, hierarchy, and mathematical computations more effectively.**

Summary

- **This unit introduced recursion, a programming technique where a method calls itself to solve smaller parts of a problem. It emphasized the importance of the base case, which stops the recursion, and the recursive case, which breaks the problem into simpler sub-problems.**
- **The unit also illustrated recursion using examples such as factorials, Fibonacci numbers, summing natural numbers, etc. It also explained how recursion uses the call stack, showing how the stack unwinds as each call returns. While recursion simplifies code, it may cause performance issues like stack overflow if not carefully managed.**